DASHFOOD

FINAL ICT PROJECT

Cafe online ordering system

SE-2622
Artur, Adiyat, Erdaulet
8 November 2026

Executive summary

DashFood is a pickup ordering system for a small cafe. Customers can check the menu and prices, add products to a cart, enter a pickup name and place an order. Staff receive that order in a separate view and update its status as it is prepared. Both views use the same PostgreSQL database. Payment takes place at pickup. The project uses React and TypeScript for the interface and Express for the API. The server checks prices and availability, saves each order in a transaction and prevents duplicate orders when a request is retried. We fixed three interface problems: hidden navigation on phones, searches with extra spaces and product forms opening outside the visible area. Automated checks cover the ordering process, staff controls, mobile layouts and database operations. The team supplied three user test times: 31, 24 and 26 seconds, all below the two-minute target. No participant reported a problem in those tasks. The charts use synthetic orders, so they demonstrate the calculations rather than real cafe demand. Long-term availability has not been established.

The report includes working-system checks and three user test results. Personal contribution signatures and long-term availability evidence are still outstanding. DashFood is the project name.

PROBLEM, SCOPE AND EVIDENCE

Many small cafes take orders at the counter or by phone. Customers may wait before they can order. Staff may also miss an item when they write an order by hand. Our system puts the menu, order details and status in one place.

Project evidence

Evidence	Supported conclusion	Limit
Annotated Stage 1, pp2-5	A documented problem, two user groups and three success criteria	No measured local queue baseline
Stage 2, final five pages	Three reproducible interface problems and instrumented page-load results	Not a timed human checkout study
Final browser/API evidence	An order reaches PostgreSQL and the staff screen; status returns to the customer	Controlled synthetic verification
User test results supplied by the team	U1: 31s; U2: 24s; U3: 26s. Disabled products blocked; no problems reported.	Dates, exact viewports and assistance were not recorded

Project scope

Customers can see the menu, check prices, change the cart and place a pickup order. Staff can process orders and edit the menu. The Insights page uses the order data for charts and a simple waiting-time rule. Payment is at the cafe. Delivery, table booking, payroll and online payments are outside our scope.

What works

The order made by the customer appears in the staff panel. Both screens use the same saved data. We have not tested the system in a real cafe, so we cannot say how much it reduces queues or increases sales.

USERS AND FUNCTIONAL REQUIREMENTS

Customers can use a phone or computer. They do not need an account. Staff sign in with the admin account. There are no separate accounts for a cashier, chef or manager. The table below shows our functional requirements.

ID	Requirement	Priority	Stage 2	Final / evidence
R1	View menu	Must	Done	Done; menu/browser check
R2	View prices	Must	Done	Done; KZT and API
R3	Add product to cart	Must	Done	Done; repeated add/feedback
R4	Remove cart product	Must	Done	Done; removal/total
R5	Change quantity	Should	Done	Done; 1-50 and reload
R6	Place order	Must	Done	Done; DB transaction, retry check
R7	View status	Should	Done	Done; customer sees Ready
R8	Staff view orders	Must	Done	Done; staff login, guest blocked
R9	Staff change status	Must	Done	Done; legal transitions only
R10	Staff add/edit products	Should	Done	Done; validated menu form
R11	Staff disable products	Unconfirmed	Inferred done	Function done; wording reconstructed

R11 is cut off in the annotated original. Its visible test says to disable a test product. We tested product disabling, but the full original sentence is missing.

Where to find the checks

The repeatable checks are tools/test-api.mjs, tools/test-browser.mjs and tools/final-checks.mjs. Dated JSON and screenshots are stored under output/evidence. The full requirement table and reasons for pending items are in docs/FINAL_REQUIREMENTS.md.

NON-FUNCTIONAL ACCEPTANCE CRITERIA

The annotated Stage 1 notes that R12-R15 lack priority and a meaningful check. The final table explicitly clarifies priority and defines observable checks without changing the original targets.

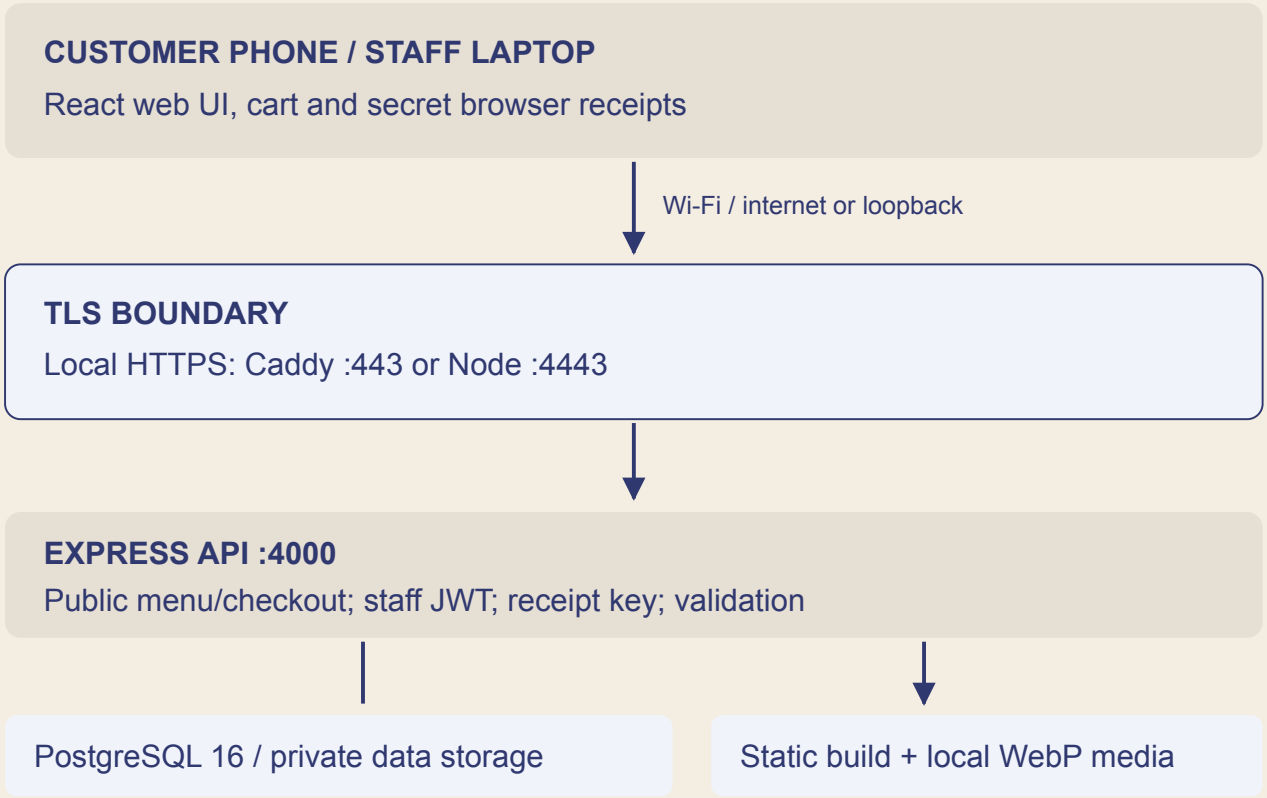
ID / priority	Target and check	Stage 2	Final status
R12 / Must	Menu and cart ready in <2s. Three cold-cache Fast 4G runs at 375x812 DPR1.	1.411s menu; 1.208s cart ready (Nov 1)	Menu 0.788s; cart 0.763s average. Local conditions only.
R13 / Must	HTTPS between customer and server; inspect trusted certificate on deployed host.	Local TLS only	Local Caddy HTTPS passes certificate checks with its local CA. Browser trust needs certificate import.
R14 / Must	No more than five main actions; count a real participant journey.	Four main actions reported	Menu, Add, Cart, name, submit. Human step-count record pending.
R15 / Must	At least 99% availability over an agreed period, including expected probes during downtime.	Unmeasured	Monitoring script supplied. Meaningful observation remains pending.

Three original success criteria

SC1: a customer places an order in less than two minutes. **SC2:** an unavailable product cannot enter a new order. **SC3:** the main ordering interface works at a width of 360 pixels. These come from Stage 1 page 5. The three important requirements on page 8 (R1, R3, R6) are a different list.

Page-load time measures the interface becoming ready. Human task time includes reading, choosing, typing and understanding the confirmation. These are different measurements. The three user checkout times were 31, 24 and 26 seconds.

AS-BUILT ARCHITECTURE



The app and PostgreSQL run on the laptop. Docker starts them in separate containers. The database has a saved volume, so its data stays after a normal restart. Express serves the website and its pictures. We do not use AWS or a native mobile app in the running version.

Boundary	Actual treatment
Public	Menu, product prices and validated guest checkout
Customer-owned	Order read requires a secret receipt token; no token in URL
Staff	Order list, menu writes and analytics require signed admin JWT
Encrypted in transit	Caddy :443 verified with local CA; Node :4443 is optional
At rest	Local PostgreSQL directory / Docker volume; disk encryption is operator configuration

HTTP :4000 remains available for local review and health checks. In public hosting, restrict direct access and serve clients through trusted HTTPS. A configured reverse proxy alone is not proof of a secure public deployment.

CHANGES FROM STAGE 1 DESIGN

Stage 1 plan	Final implementation	Reason
React web + React Native	Responsive React web	A 360px browser journey satisfies the stated mobile scope without a second app.
Gateway / Nginx	Express static server; optional Caddy TLS	One origin simplifies local review; Caddy handles domain certificates.
AWS EC2, RDS and S3	Local PostgreSQL / Docker volume and bundled images	Reproducible course demo; no claimed cloud resources or cloud pricing.
UUID products/categories, user-owned paid orders	Serial IDs, fixed categories, guest pickup orders	Smaller schema for the implemented pickup workflow; payment occurs in person.
Stock quantities and product FK in order items	Availability flag and item name/price snapshots	Scope requires unavailable-product rejection and historical totals, not inventory.
AES-256 storage and universal TLS claim	Actual local storage and explicit optional TLS	Describe verified controls and configuration limits.
General future smart technology	Live order-count wait rule	A transparent simulation reuses existing data; hardware remains future work.

Cost and performance justification

We use our laptop and open-source software for the demo. We do not pay for a cloud server. A real cafe would still need to cover hosting, internet and maintenance costs. The old AWS free-tier statement is not used in this report.

One app process and one database are enough for this project demo. The server reuses database connections. Indexes help it find order items and dates. Smaller WebP pictures reduce download size. We have not tested 10,000 users at the same time.

DATA MODEL AND ORDER RECORDS

Table	Important columns	Purpose / relationship
users	id, username, password_hash, role	Unique staff account; bcrypt password hash
products	id, name, category, price, description, active	Whole positive KZT; fixed category check
orders	id, total, status, created_at, customer_name, is_demo	One pickup transaction and its current state
orders: final additions	tracking_hash, request_key, request_hash	Customer read authorization and retry deduplication
order_items	id, order_id, name, price, qty	Many items per order; FK to orders with cascading delete
seed_batches	name, created_at	Marks one repeatable synthetic seed batch

Checkout transaction

The API checks the name, product IDs and quantities. Each checkout has a request key. A database advisory lock makes requests with the same key wait for each other. If that order already exists, the server returns it. A different cart with the same key is rejected. The server reads active product rows with a shared lock, obtains their names/prices and calculates the total.

It inserts orders and order_items, commits them together, then returns the receipt. Failures roll back the transaction. Row locks prevent a concurrent menu edit from changing price or availability midway through this write. A unique index is an additional guard on request_key.

Historical correctness

Each order item retains the purchased name, price and quantity. Editing a menu price later does not rewrite an old receipt. Integer KZT avoids floating-point currency rounding in this scope. Quantity is limited to 50 per product and the total to 10,000,000 KZT. Parameterized queries treat customer text as data.

The schema is additive for existing installations. Legacy orders without a tracking hash remain staff-visible but cannot be opened publicly by guessing their IDs. No old customer tracking credentials are fabricated.

DATA SECURITY

Risk	Implemented control	Remaining limit
Staff impersonation	bcrypt hashes, HS256 JWT verification, eight-hour expiry, login rate limit	No MFA or managed account administration
Guessing another order	Secret token returned at checkout; only SHA-256 token hash stored; constant-time comparison	Browser storage must be retained and protected
Price manipulation	Prices read from DB; customer-supplied total ignored	No inventory reservation or online payment
Double checkout after lost response	Persistent checkout key and serialized server retries	Different keys represent different orders
Malformed input or SQL injection	JSON size cap, type/range checks, parameterized SQL, valid identifiers	No claim of a complete penetration test
Invalid staff actions	Atomic state transition guards; terminal statuses cannot jump backwards	One admin role, no detailed staff audit log

Personal data and location

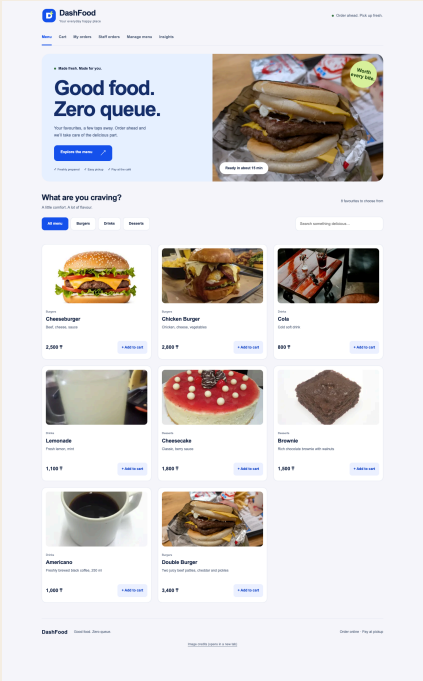
Pickup names are stored in the PostgreSQL orders table and shown only to authenticated staff. Customer order read omits the name. The application collects no phone, address or payment-card details. In this review, DB files reside on the laptop under .local; in Docker they reside in a named volume on the deployment host. No cloud location is assumed.

Before real use, the operator needs a documented hosting location, retention/deletion policy, encrypted backups and an appropriate review of Kazakhstan personal-data obligations. Local synthetic testing does not establish legal compliance. The final build does not claim automatic AES-256 encryption of database files.

Visible recovery

If checkout fails, the cart stays saved. The customer can remove an unavailable product and try again. A retry keeps the same request key. If one saved order cannot be opened, the other orders still load. Staff sign in again when their session expires. An unknown API address returns a 404 error.

CUSTOMER INTERFACE AND MULTIMEDIA



Final production menu captured during the database-backed browser journey.

The menu shows product names, prices, categories and descriptions. Customers can search for food and add it to the cart. The cart shows quantity and total price. They can remove a product or change its quantity from 1 to 50. Before sending the order, they enter a pickup name.

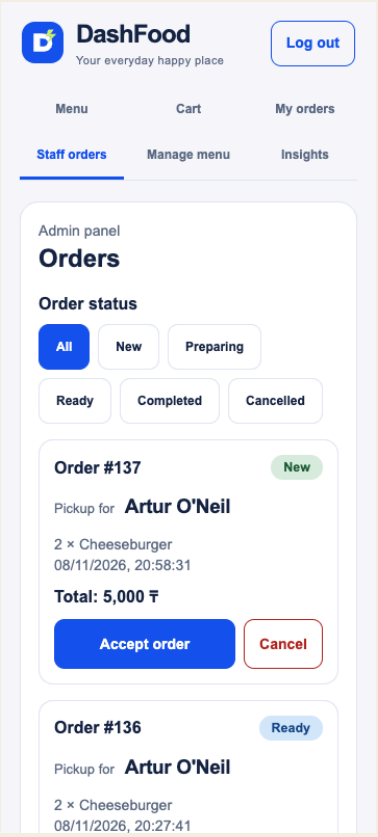
Media and accessibility decisions

We use WebP photos in different sizes. The browser chooses a suitable size through srcSet. The main photo loads first. Other photos can load when the customer scrolls down. Each photo has alternative text. Image sources are listed on the credits page. The photos are examples, not photos of a real partner cafe.

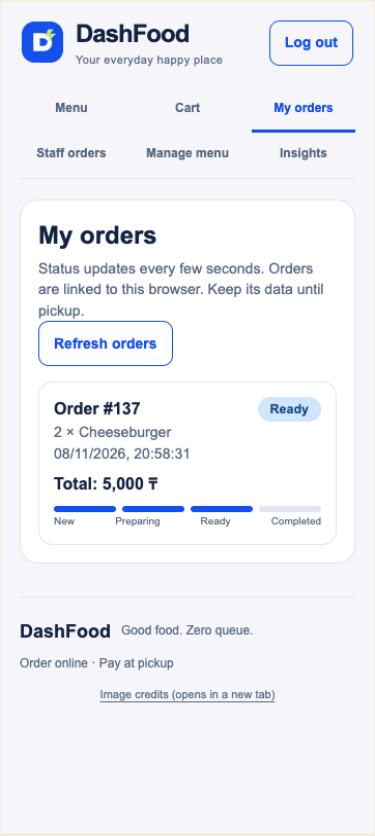
Main buttons have minimum heights of 44 or 48 CSS pixels. Keyboard focus is visible. Reduced-motion preference disables animations and smooth scrolling. Text appears on opaque surfaces rather than photographs. Brand text #303870 on white has a calculated contrast of 5.95:1; secondary text #303870 on white is 5.46:1. Ratios use the WCAG sRGB luminance formula.

> Automated accessibility scanning covers six screens at 360px. It cannot certify every screen-reader experience. User-controlled backup video is a separate presentation artifact; the application plays no automatic audio.

CUSTOMER AND STAFF ORDERS



Authenticated staff receive the saved pickup order.



Customer reads Ready after the staff action.

Staff sign in to see the same PostgreSQL order created by the customer. The required workflow is New, Preparing, Ready and Completed. Cancellation is allowed from New or Preparing. Attempts to skip stages or update an already changed order are rejected. Busy controls reduce accidental repeated actions.

Manage menu supports creating a product, editing name/description/category/whole KZT price and toggling availability. An unavailable product disappears from public menu responses and cannot be purchased from a stale cart. Editing the last product scrolls to the form and focuses its Name field; Cancel restores the original Edit button.

Two user groups in one demonstration

In the demo, we make an order as a customer and then open it as staff. We show the same order number in the staff panel and in the database. This shows how the two user groups work with the same data.

FINAL TESTING AND STAGE 2 COMPARISON

Success criterion	Target	Stage 2	Final / met?
SC1: place an order	<120s human task	Wrong task labels; no human duration	U1: 31s; U2: 24s; U3: 26s. All below 120s; mean 27s.
SC2: unavailable product	No unavailable order accepted	API rejection reported	All three participants reported rejection. The API checks also pass.
SC3: mobile interface	360px full journey	375px evidence on last slides	Automated 360px journey passes. Participants reported a working interface; their exact widths were not recorded.

System checks

53 API assertions passed. They include staff authorization, invalid data, DB pricing, transaction results, workflow guards, disabled products, secret tracking and five concurrent retry requests creating exactly one order. The browser test finished without JavaScript errors.

6 accessibility scans cover menu, cart, login, staff orders, product editor and Insights at 360px. The final run detected 0 WCAG A/AA violations in that automated scope. These results are dated 2026-11-08 and use 157.0.8103.38.

User test results

Participant	Device / browser	Order time	Reported result
U1	iPhone / Safari	31 seconds	No problems; disabled product blocked; interface worked
U2	Windows PC / Chrome	24 seconds	No problems; disabled product blocked; interface worked
U3	Android / Chrome	26 seconds	No problems; disabled product blocked; interface worked

The team supplied these results. The mean checkout time was 27 seconds and all three were below the 120-second target. Test dates, exact viewport widths and any assistance were not recorded. A working interface on a phone does not by itself confirm the exact 360px criterion. No problems were reported in these short tasks; this does not establish that the application is bug-free.

CHANGES AFTER TESTING AND PAGE SPEED

Finding from Stage 2	Final correction	Repeatable result
F01: right-hand routes hidden on mobile	All six navigation actions appear in a grid at narrow widths	All nav button bounds fit at 320/360/375/768/1440px
F02: padded search returns no result	Trim before matching, whitespace-only query is empty search	Padded Cheeseburger finds one card; spaces restore list
F03: form opens above the viewport	Immediate scroll/focus to Name; restore focus on Cancel	Last product form visible/focused at 360px; return focus verified

Production-build measurement

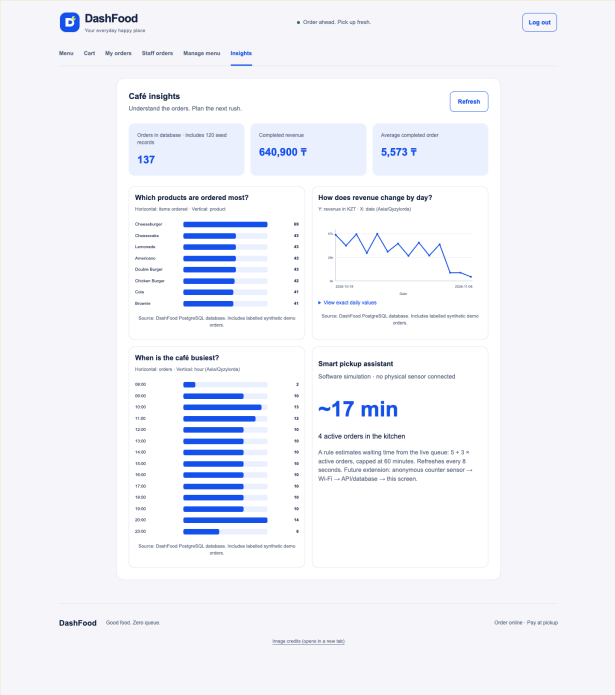
Screen	Ready run 1	Run 2	Run 3	Mean ready	Mean load
menu	0.799s	0.782s	0.784s	0.788s	0.546s
cart	0.763s	0.753s	0.774s	0.763s	0.542s

Conditions: fresh browser contexts, cache disabled, 375x812 CSS pixels, DPR1, 1,012,500 bytes/s down, 168,750 bytes/s up and 165ms latency through Chrome DevTools Protocol. Ready means product cards or cart quantity controls become observable. It is not human task completion. The JSON retains each run and browser version.

Stage 2 reported 1.411s menu and 1.208s cart readiness on November 1. The final run uses a changed build and a different session; the comparison is descriptive, not a controlled causal performance-improvement claim. App screenshots are not screenshots of the Network panel. Manual Network captures remain a separate evidence task if required by the instructor.

R12 is assessed only under the recorded local test conditions. Internet distance, host resources, caching and device speed can change the result.

DATA ANALYSIS AND INTERPRETATION



Actual Insights capture. The analysis snapshot is synthetic course demonstration data.

Question	Query / unit	Current captured result
Which items are ordered most?	SUM(item.qty), excluding Cancelled; units	Leader: Cheeseburger (69)
How does revenue vary by day?	SUM(order.total) for Completed, local date; KZT	Completed total 640,900 KZT
When are requests busiest?	COUNT(*) grouped by local hour; orders	Peak 20:00

The snapshot contains 137 orders, of which 120 are flagged seed records. Remaining records in this assessment were also created by verification, not paying customers. Average completed order: 5,573 KZT. Capture time: 2026-11-08T16:11:43.485Z.

The queries join orders with their items and use SUM, COUNT and GROUP BY. They show product quantities, completed revenue and busy hours. Our data is made for testing. A real cafe would need its own sales records before using the charts to plan staff or stock.

SMART RULE AND ITS LIMITS

Implemented simulation

Insights calculates an estimated waiting time using **min(60, 5 + 3 x active orders)** minutes. Active means New or Preparing. The screen refreshes every eight seconds and explicitly states that no physical sensor is connected. Staff actions change the queue and therefore the estimate.

Active orders	Calculated estimate	Meaning
0	5 minutes	Fixed baseline
3	14 minutes	5 + 3 x 3
20	60 minutes	Uncapped 65 reaches the 60-minute limit

This is a simple rule, not an AI prediction. We have not compared it with real kitchen preparation times. Staff can use it as a demo of how the queue changes, but it is not an exact pickup promise.

Future anonymous occupancy counter

A possible extension uses a time-of-flight pickup counter, Wi-Fi with per-device credentials, authenticated HTTPS ingestion and a sensor_readings table. The staff screen would combine aggregate occupancy with the current queue. No camera, face recognition or phone identifiers are needed for this proposed design.

If the device sends one reading every ten seconds, it sends 8,640 readings per day. Assuming 100 bytes of JSON per reading gives about 0.864 MB of raw JSON, excluding network overhead, retries and storage indexes. A future retention proposal is seven days for raw data and 30 days for minute aggregates. These are design assumptions, not deployed collection or retention policies.

What can go wrong

If the sensor stops sending data, the screen should show a warning. The server should reject repeated readings and impossible counts. Even without names, the data can show when people are present. This sensor extension has not been built or tested.

RUNNING THE APP AND KEEPING DATA

One local review command

Install dependencies with npm run setup, then run npm run review. It compiles the app, starts a persistent local PostgreSQL 16 instance, applies additive schema changes, inserts the seed batch once and starts HTTP/TLS endpoints. StartPreview.cmd provides the Windows entry point. The README lists the guest and staff role credentials and where generated passwords live.

Alternative Docker path

Open Docker Desktop and run setup.ps1 on Windows or bash setup.sh on macOS/Linux. App and database health checks verify the connection. A named volume persists data across ordinary stops. The optional Caddy profile accepts an owned domain for HTTPS. We also tested the final Docker version. The API, customer and staff screens worked, and the order data stayed after stopping and starting the app and database.

Backup and restore exercise

For Docker, an operator can export with docker compose exec -T db pg_dump -U cafe -d cafe and save the output securely. Rehearse restoration into a separate test database before relying on a backup. A backup is sensitive because it contains pickup names. Do not publish it or store it in Git. Ordinary restart must preserve existing orders.

Availability measurement

tools/monitor.mjs records health responses, latency and a fixed observation schedule; restarting resumes the saved run and counts missed scheduled probes as unavailable. A meaningful R15 assessment needs a declared duration and inclusion of missed probes during downtime. Restart policies and a successful launch demonstrate recovery mechanisms; they do not by themselves establish 99% uptime.

Repository evidence

Existing history remains intact. The latest integrated changes are in the local main working folder and still need to be committed and pushed. GitHub Actions is configured to run build, API and browser/accessibility checks using PostgreSQL 16. Check the GitHub Actions result after pushing the final changes.

LIMITATIONS AND FINAL ACTION REGISTER

Open item	Required evidence / next action	Owner for confirmation
User test details	Add test dates and any assistance if the participants can confirm them	Testing facilitator
Exact 360px user test	Record the viewport width in a further user session; product rejection is already reported	Testing facilitator
R11 exact wording	Obtain the uncropped original or confirm wording with instructor	Whole team
Trusted HTTPS	Show valid deployed certificate and close direct public HTTP exposure	Deployment reviewer
R15 >=99% availability	Declare observation period; collect and interpret health probes	Deployment reviewer
B2 contribution statements	Each member writes/verifies 100-120 words with files, problem, reflection, signature	Artur / Adiyat / Erdaulet
Individual Git history	Review each own change, commit with own account, retain honest history	Each member
Final presentation/LMS	Rehearse 8-10 minutes; each member speaks; upload once before course deadline	Whole team

Future engineering work

Before using the app in a real cafe, we need a clear rule for deleting old customer data, tested backups and regular monitoring. If the cafe gets many orders, the staff list should load in pages. We would also test how many requests the server can handle.

The app does not include delivery, stock counts or card payments. These were outside the original pickup project. Our next step is to finish the remaining records and practise the demo.

The tables show which requirements are done and which still need evidence. Each member must also be able to explain the code they worked on.

TEAM WORK AND DEFENSE

We worked on the project together. The roles in Stage 1 were our plan. Each member needs to describe their actual work and link the files or commits that show it. Existing commits stay under their original authors.

Member	Specific final review focus	Personal evidence to add
Artur	Customer interface, mobile layout and menu/cart usability	Own file/commit, solved problem, reflection and signature
Adiyat	API, data model, query/transaction behaviour and deployment boundaries	Own file/commit, solved problem, reflection and signature
Erdaulet	Correct success criteria, instructor feedback, evaluation and defense package	Review final docs, repeat checks, then own final commit

Statement format: 100-120 words per member

Each member should state exactly what they personally built or verified, link to the corresponding real files/commits, explain one problem and how they solved it, and name one improvement for next time. Sign only after reviewing that statement. docs/CONTRIBUTIONS.md provides a worksheet. Signed statements are pending; each person must sign their own statement.

Erdaulet commit preparation

The requested identity is Erdaulet <q7998378@gmail.com>, GitHub q7998378-svg. After personal review, the final evaluation changes can be committed with that identity. The account email must be associated with GitHub for profile attribution. A final commit proves the current contribution, not all previous team development.

Individual defense readiness

Each member needs to explain both user journeys and the full browser/API/database path. Practice adding a validated form field, modifying a SQL aggregate and finding a specific commit. The Russian defense guide maps exact files and provides a nine-minute shared presentation plan and 3-5-minute question practice.

REFERENCES AND SUBMISSION CHECKLIST

Reference	Use in this report
ICT_Final (1).pdf, supplied course brief	Final A-D criteria, 12-20 pages, 8-10 minute presentation, personal defense
midterm_ANNOTATED.pdf, supplied Stage 1	Original success criteria p5, requirements p7, instructor annotations
ICT_EndtermAdiyatEraArtur.pdf, supplied Stage 2	Prior results and internal findings F01-F03; final five pages
github.com/Arqode/CafeSystem	Original project and preserved history; baseline 7a39fcf
PostgreSQL 16: Transactions; Explicit Locking	Transaction/row/advisory lock concepts used in checkout
Express: Production Security	TLS, input handling and dependency hygiene
Vite: Getting Started	Production client build and supported runtime
Embedded PostgreSQL project	Local PostgreSQL distribution/runtime setup
web/public/media/credits.html and provenance.json	Project image sources and credits
output/evidence/*.json and screenshots, 8 Nov 2026	Current API, UI, accessibility, performance and dataset evidence

Technical references accessed 8 November 2026. Course textbooks remain defense preparation material.

Before submission

Confirm the missing user test details and R11, complete and sign individual statements, verify each own commit, supply trusted TLS and availability evidence, rehearse all speakers, and upload one team package under the required name. Check the submission deadline in the syllabus.

Report name: SE2622_DashFood_ICT_FinalProject.pdf. DashFood fills the project identifier because the group has no separate team name. The limitations and remaining tasks are listed in the report.